

Order and Order Details Management System

Scenario: An e-commerce enterprise handles customer transactions. The database architecture must store customer orders, track individual order line details, and maintain relational integrity across customers, orders, and products.

1. Create Orders and Order Details Tables:

```
SQL> CREATE TABLE ORDERS (Order_ID VARCHAR2(10), Customer_ID VARCHAR2(10), Order_Date DATE, Total_Amount NUMBER(10,2), Order_Status VARCHAR2(20));
Table created.

SQL> CREATE TABLE ORDER_DETAILS (Order_Detail_ID VARCHAR2(10), Order_ID VARCHAR2(10), Product_ID VARCHAR2(7), Quantity NUMBER, Price NUMBER(10,2));
Table created.
```

2. Establish Relationships & Apply Constraints:

Customer and Orders:

- One customer can place multiple orders, but each order belongs to one customer.
- Relationship: **Customer (1) — (Many) Orders**

Orders and Products:

- One order can contain multiple products, and one product can appear in multiple orders.
- Relationship: **Orders (Many) — (Many) Product** (implemented via the **Order_Details** table).

Primary Key Constraints:

- Assign Primary Key constraint to ORDERS table on Order_ID.
- Assign Primary Key constraint to ORDER_DETAILS table on Order_Detail_ID.

```
SQL> ALTER TABLE ORDERS ADD CONSTRAINT PK_Orders PRIMARY KEY (Order_ID);
Table altered.

SQL> ALTER TABLE ORDER_DETAILS ADD CONSTRAINT PK_OrderDetails PRIMARY KEY (Order_Detail_ID);
Table altered.
```

Foreign Key Constraints:

- Foreign Key on ORDERS pointing to Customer_ID in CUSTOMER table.
- Foreign Key on ORDER_DETAILS pointing to Order_ID in ORDERS table.
- Foreign Key on ORDER_DETAILS pointing to Product_ID in PRODUCT table.

```
SQL> ALTER TABLE ORDERS ADD CONSTRAINT FK_Orders_Customer FOREIGN KEY (Customer_ID) REFERENCES CUSTOMER(Customer_ID);
Table altered.

SQL> ALTER TABLE ORDER_DETAILS ADD CONSTRAINT FK_OD_Orders FOREIGN KEY (Order_ID) REFERENCES ORDERS(Order_ID);
Table altered.

SQL> ALTER TABLE ORDER_DETAILS ADD CONSTRAINT FK_OD_Product FOREIGN KEY (Product_ID) REFERENCES PRODUCT(Product_ID);
Table altered.
```

NOT NULL Constraint:

- Apply NOT NULL condition to Order_Date field.

```
SQL> ALTER TABLE ORDERS MODIFY Order_Date NOT NULL;  
Table altered.
```

Check Constraints:

- Set Check constraint to ensure Total_Amount remains non-negative (≥ 0).
- Set Check constraint to restrict Order_Status to allowed values ('Pending', 'Shipped', 'Delivered').
- Set Check constraint to ensure Quantity is positive (> 0).
- Set Check constraint to ensure Price is non-negative (≥ 0).

```
SQL> ALTER TABLE ORDERS ADD CONSTRAINT CHK_Total CHECK (Total_Amount >= 0);  
Table altered.  
SQL> ALTER TABLE ORDERS ADD CONSTRAINT CHK_Status CHECK (Order_Status IN ('Pending', 'Shipped', 'Delivered'));  
Table altered.
```

```
SQL> ALTER TABLE ORDER_DETAILS ADD CONSTRAINT CHK_Qty CHECK (Quantity > 0);  
Table altered.  
SQL> ALTER TABLE ORDER_DETAILS ADD CONSTRAINT CHK_OD_Price CHECK (Price >= 0);  
Table altered.
```

Insert Values into ORDERS Table:

```
SQL> INSERT INTO orders VALUES ('ORD001', 'CUS001', DATE '2026-09-01', 520.00, 'Delivered');  
1 row created.  
SQL> INSERT INTO orders VALUES ('ORD002', 'CUS002', DATE '2026-09-02', 250.00, 'Delivered');  
1 row created.  
SQL> INSERT INTO orders VALUES ('ORD003', 'CUS003', DATE '2026-09-03', 600.00, 'Shipped');  
1 row created.  
SQL> INSERT INTO orders VALUES ('ORD004', 'CUS004', DATE '2026-09-04', 400.00, 'Delivered');  
1 row created.
```

```
SQL> SELECT * FROM ORDERS;
```

ORDER_I	CUSTOME	ORDER_DAT	TOTAL_AMOUNT	ORDER_STATUS
ORD001	CUS001	01-SEP-26	520	Delivered
ORD002	CUS002	02-SEP-26	250	Delivered
ORD003	CUS003	03-SEP-26	600	Shipped
ORD004	CUS004	04-SEP-26	400	Delivered

Insert Values into ORDER DETAILS Table:

```
SQL> INSERT INTO ORDER_DETAILS VALUES ('OD001', 'ORD001', 'PR0001', 1, 200.00);
1 row created.

SQL> INSERT INTO ORDER_DETAILS VALUES ('OD002', 'ORD001', 'PR0002', 1, 320.00);
1 row created.

SQL> INSERT INTO ORDER_DETAILS VALUES ('OD003', 'ORD002', 'PR0003', 1, 250.00);
1 row created.

SQL> INSERT INTO ORDER_DETAILS VALUES ('OD004', 'ORD003', 'PR0006', 1, 600.00);
1 row created.

SQL> INSERT INTO ORDER_DETAILS VALUES ('OD005', 'ORD004', 'PR0005', 1, 400.00);
1 row created.
```

```
SQL> SELECT * FROM ORDER_DETAILS;
```

ORDER_D	ORDER_I	PRODUCT	QUANTITY	PRICE
OD001	ORD001	PR0001	1	200
OD002	ORD001	PR0002	1	320
OD003	ORD002	PR0003	1	250
OD004	ORD003	PR0006	1	600
OD005	ORD004	PR0005	1	400

Updates and Deletions:

```
SQL> UPDATE orders SET total_amount = 520.00, order_status = 'Shipped' WHERE order_id = 'ORD010';
1 row updated.
```

```
SQL> DELETE FROM orders WHERE order_id = 'ORD011';
1 row deleted.
```

Report 1: Customer Order History

Display:

- Customer Name
- Order ID
- Order Date
- Total Amount

```
SQL> SELECT CUSTOMER_NAME, ORDER_ID, ORDER_DATE, TOTAL_AMOUNT FROM CUSTOMER JOIN ORDERS ON CUSTOMER.CUSTOMER_ID = ORDERS.CUSTOMER_ID;
```

CUSTOMER_NAME	ORDER_I	ORDER_DAT	TOTAL_AMOUNT
Sanjay Kumar	ORD001	01-SEP-26	520
Harini Devi	ORD002	02-SEP-26	250
Praveen Raj	ORD003	03-SEP-26	600
Keerthana S	ORD004	04-SEP-26	400
Pavithra M	ORD006	06-SEP-26	850
Ashwin Anand	ORD007	07-SEP-26	420
Lavanya M	ORD010	10-SEP-26	320
Surya Prakash	ORD009	09-SEP-26	200
Nithya R	ORD008	08-SEP-26	360
Dinesh Kumar	ORD005	05-SEP-26	180

```
10 rows selected.
```

Report 2: Product-wise Order Report

Find: Number of times each product was ordered and total quantity sold.

```
SQL> SELECT P.PRODUCT_NAME, SUM(O.QUANTITY) AS TOTAL_QUANTITY FROM PRODUCT P JOIN ORDER_DETAILS O ON P.PRODUCT_ID = O.PRODUCT_ID GROUP BY P.PRODUCT_NAME;
```

PRODUCT_NAME	TOTAL_QUANTITY
Organic Oats	1
Steel Lunch Box	1
Bamboo Tray	1
Cotton Tote Bag	3
Aloe Face Cream	1
Eco Cotton Kurta	1
Eco Diary	3
Natural Honey	2

8 rows selected.

Report 3: Customer Purchase Analysis

- **Customers with maximum orders:**

```
SQL> SELECT c.Customer_Name, COUNT(o.Order_ID) AS Order_Count FROM CUSTOMER c JOIN ORDERS o ON c.Customer_ID = o.Customer_ID GROUP BY c.Customer_Name HAVING COUNT(o.Order_ID) = (SELECT MAX(COUNT(Order_ID)) FROM ORDERS GROUP BY Customer_ID);
```

CUSTOMER_NAME	ORDER_COUNT
Sanjay Kumar	1
Ashwin Anand	1
Lavanya M	1
Nithya R	1
Praveen Raj	1
Surya Prakash	1
Dinesh Kumar	1
Keerthana S	1
Harini Devi	1
Pavithra M	1

10 rows selected.

- **Total spending by each customer:**

```
SQL> SELECT c.Customer_Name, SUM(o.Total_Amount) AS Total_Spent FROM CUSTOMER c JOIN ORDERS o ON c.Customer_ID = o.Customer_ID GROUP BY c.Customer_Name;
```

CUSTOMER_NAME	TOTAL_SPENT
Sanjay Kumar	520.00
Ashwin Anand	420.00
Lavanya M	520.00
Nithya R	360.00
Praveen Raj	600.00
Surya Prakash	200.00
Dinesh Kumar	180.00
Keerthana S	400.00
Harini Devi	250.00
Pavithra M	850.00

10 rows selected.

- **Average order value:**

```
SQL> SELECT AVG(Total_Amount) AS Average_Order_Value FROM ORDERS;
```

AVERAGE_ORDER_VALUE
430